LEKTRA

High-performance Document and Image viewer that prioritizes screen space and control.

Important Links

Homepage

https://dheerajshenoy.github.io/lektra

Configuration reference

https://dheerajshenoy.github.io/lektra/configuration.html

Commands reference

https://dheerajshenoy.github.io/lektra/commands.html

Source (Codeberg)

https://codeberg.org/lektra/lektra

Source (GitHub mirror)

https://github.com/dheerajshenoy/lektra

> **Note:** This guide assumes you have installed **LEKTRA** and have not changed the default keybindings.

Contents

1 Getting Started

1.1 Opening a file

You can open a document in multiple ways:

- **Command line:** `lektra <path-to-file>`
- **File explorer:** Right-click a file and choose Open with lektra
- **From within lektra:** Press `o` to open the file picker

Supported formats include PDF, EPUB, XPS, CBZ, MOBI, FB2, and common image formats (JPG, PNG, APNG, WEBP, TIFF, GIF, BMP, ICO, SVG, PPM, PGM, PBM). DjVu, high-quality SVG rendering (via librsvg), and EXIF metadata are detected at runtime — install `libdjvulibre21`, `librsvg2`, and `libexif12` respectively for those features; LEKTRA continues to work without them.

You can also:

- Open multiple files at once — each opens in its own tab
- Open a file in a new window with `:open_file_new_window`
- Drag and drop files from your file manager onto the LEKTRA window
- Use `:files_recent` (or the File → Recent Files menu) to reopen a previously viewed document at the last page you were on

When `behavior.auto_reload` is enabled in `config.toml`, LEKTRA watches the open documents on disk and reloads them automatically when they change — useful for `latexmk` workflows or any tool that re-exports the PDF. With auto-reload disabled, use `:file_reload` to reload manually.

First run: On the first launch LEKTRA shows a support dialog with donation links (Ko-fi, Liberapay, GitHub Sponsors). You can reopen it any time from Help → Donate / Support or via `:donate`. LEKTRA is free forever — the dialog is entirely optional.

1.2 Tabs and Splits

LEKTRA supports tabs and splits for viewing multiple documents simultaneously.

- `o` opens a file in a new tab
- `Alt+1` … `Alt+9` jumps directly to tab 1 through 9
- Cycling and closing tabs have no default keybinding — bind `tab_next`, `tab_prev`, `tab_close` in your config, or invoke them from the `:` command palette

Splits use Vim-style `Ctrl+W` chords:

- `Ctrl+W,s` splits horizontally (`split_horizontal`)
- `Ctrl+W,v` splits vertically (`split_vertical`)
- `Ctrl+W,h` / `Ctrl+W,j` / `Ctrl+W,k` / `Ctrl+W,l` moves focus left / down / up / right
- `Ctrl+W,c` closes the current split

Splits are independent views — each remembers its own zoom, page position, and fit mode. When `split.focus_follows_mouse` is enabled in the config, hovering over a pane also gives it focus.

Tabs across windows: You can drag a tab out of the tabbar to detach it into its own window, or drop it onto another LEKTRA window to move it there.

1.3 Keyboard Navigation and Keybindings

The core philosophy of **LEKTRA** is keyboard-driven navigation (mouse support is available too). Default keybindings follow VIM conventions.

1.3.1 Scrolling and Movement

- **Vertical:** `j` scrolls down, `k` scrolls up
- **Horizontal:** `h` scrolls left, `l` scrolls right
- `Ctrl+d` / `Ctrl+u` — scroll down / up half a page

1.3.2 Page Navigation

- `Shift+j` moves to the next page
- `Shift+k` moves to the previous page
- `g,g` (chord) jumps to the first page
- `Shift+g` jumps to the last page
- `Ctrl+g` prompts for a page number and jumps to it

1.3.3 Search

- `/` opens or focuses the search bar
- `n` jumps to the next search result
- `Shift+n` jumps to the previous search result

- `:search_regex` opens search with regex mode primed
- `:search_below term` / `:search_above term` search only pages after / before the current page (one-shot; the next plain search reverts to whole-document scope)
- `:picker_annot_comment_search` finds text inside annotation comments and popups

1.3.4 Narrow (Focused Reading)

You can restrict the viewport, search, and scrolling to a subset of the document:

- `:narrow_to_region` — first enter region-selection mode (`4`) and drag out a rectangle, then run this command; only that region stays visible, the rest dims
- `:narrow_to_pages 5 10` — restrict to a page range (also accepts `5-10`)
- `:narrow_to_section` — restrict to a section by outline title (fuzzy match, picker if no argument)
- `:widen_region` — exit narrow mode

While narrowed, an orange **N** badge appears in the statusbar; hover it for a reminder of how to exit. Search stays inside the narrowed range and rotation/flip remap the region correctly.

1.3.5 Zoom and View Control

- `=` zooms in, `-` zooms out, `0` resets zoom

Fit modes:

- `Ctrl+Shift+W` fit to width
- `Ctrl+Shift+H` fit to height
- `Ctrl+Shift+=` fit to window
- `Ctrl+Shift+R` toggle auto-resize

1.3.6 Layout Modes

For multi-page documents you can pick how pages are arranged:

- **Single** — one page at a time
- **Continuous vertical** — pages stacked vertically (default for PDFs)
- **Continuous horizontal** — pages side by side
- **Book** — two-page facing spread with a cover offset (for magazines, novels, etc.)

Set a layout directly with one of `:layout_single`, `:layout_vertical`, `:layout_horizontal`, `:layout_book`, or use the View → Layout menu. The default at startup is controlled by `layout.mode` in `config.toml`.

1.3.7 Reading Modes

- `Ctrl+Shift+M` — toggle the menubar (`menubar`)
- `:fullscreen` — toggle full-screen (no default keybinding)
- `:presentation_mode` — presentation mode (hides all chrome, one page at a time)
- `:focus_mode` — focus mode (hides menubar/tabbar/statusbar while keeping the split UI)
- `:tabs`, `:statusbar` — toggle the tabbar and statusbar

1.3.8 Outline and History

- `t` opens the document outline (table of contents) overlay
- `Alt+Shift+H` opens the highlight annotation search overlay
- `Ctrl+o` goes back to the previous location in history
- `Ctrl+i` goes forward in the location history

If the PDF has no embedded outline, LEKTRA can build one from the document text by scanning font sizes: run `:generate_outline` and the result appears in the outline picker. Outlines can also be exported to (`:export_outline`) and loaded from (`:load_outline`) a portable JSON file, so you can share a hand-crafted TOC between machines.

1.3.9 Bookmarks and Marks

Two independent mechanisms for jumping around:

- **Bookmarks** are persistent, named per-document markers. Add one with `:bookmark_add`, open the picker with `:bookmarks`, or manage them from the Bookmarks menu. Bookmarks survive between sessions.
- **Marks** are single-letter jump points inside the current document, Vim-style. Press `:mark_set` then a letter to set, `:mark_goto` then a letter to jump back. `:mark_delete` clears one. Marks live only for the current session.

1.3.10 Annotations and Interaction Modes

> **Note:** These keys toggle modes. Some modes require clicking or dragging on the page to take effect.

- `1` text-selection mode
- `2` highlight-annotation mode

- `3` rectangle-annotation mode
- `4` region-selection mode
- `5` popup-annotation mode

1.3.11 Links and Actions

- `f` link-hint visit (keyboard link navigation)
- `o` open file in a new tab
- `Ctrl+Shift+O` open file picker (`file_picker`)
- `Alt+Shift+O` recent files picker (`files_recent`)
- `Ctrl+S` save current document

1.3.12 Editing and UI

- `u` undo, `Ctrl+R` redo
- `v` visual-line mode
- `i` invert colors
- `Ctrl+Shift+M` toggle menubar
- `:` open command palette

1.3.13 Rotation and Flip

- `<` rotate counter-clockwise, `>` rotate clockwise
- `|` flip horizontal, `_` flip vertical

1.4 Sessions

A session is a snapshot of your open tabs, splits, and per-view state (page, zoom, layout, rotation). Sessions let you switch between reading projects without losing your place.

- `:session_save` — save the current layout to the named session (or the active one)
- `:session_save_as` — save under a new name
- `:session_load` — pick a session from the list and restore it

Sessions live under `$HOME/.local/share/lektra/sessions/` (Linux/macOS) as JSON files, one per session.

1.5 Portal Mode

Portal mode links two views to the same document so that scrolling one drives the other — useful for cross-referencing figures with body text, or keeping a table of contents pane in sync with the main view. Trigger with `:portal`.

1.6 File Properties

`:file_properties` (or File → File Properties) opens the properties dialog. For PDFs it shows the info dictionary (title, author, producer, creation date, PDF version, encryption status, page count). For images it shows width, height, format, animation status, and — when `libexif` is installed — full EXIF metadata (camera model, lens, ISO, shutter, aperture, GPS, date taken).

1.7 Copying and Exporting

- `:selection_copy` — copy the current text selection to the clipboard
- `:copy_page_image` — copy the current page as an image (context menu also has "Copy Region as Image" when a region is selected, with an optional custom DPI for vector documents)
- `:export_outline` — write the document outline (real or generated) to JSON
- `:file_save_as` — save the current PDF to a new location (including a copy of any annotations you have added)

1.8 Encrypted PDFs

Password-protected PDFs prompt for a password on open. You can also encrypt or decrypt a PDF from within LEKTRA:

- `:file_encrypt` — add a password to the current document
- `:file_decrypt` — remove the password (requires the current one)

1.9 Command Palette and Pickers

Press `:` to open the command palette — a fuzzy-matching prompt that runs any LEKTRA command by name. Beyond commands, several dedicated pickers exist:

- `:command_palette` — list every command with its description
- `:file_picker` — open a file
- `:files_recent` — jump to a recently opened file, restored to its last page

- `:picker_outline` — outline / TOC
- `:bookmarks` — bookmarks
- `:picker_highlight_search` — search across highlight annotations

2 Configuration

LEKTRA can be configured using **TOML** or **Lua**. Both files are optional and both can be present; the TOML file is applied first, and the Lua file (`$HOME/.config/lektra/init.lua`) is sourced afterwards so it can override any TOML setting or add runtime behaviour.

Let us look at how to configure using TOML first. The configuration file should be at `$HOME/.config/lektra/config.toml`. Run `:open_config` to open whichever config file(s) exist — a chooser appears if both are present.

2.1 Key sections

`[window]`
Window size, fullscreen, menubar visibility, title format, accent color.

`[page]`
Page foreground and background colors.

`[statusbar]`
Visibility and which fields to display (page number, progress, file info, etc.).

`[split]`
Split behaviour — focus follows mouse, dim inactive panes, etc.

`[portal]`
Picture-in-picture portal overlay settings.

`[annotations.highlight] / [annotations.rect] / [annotations.popup]`
Per-annotation-type appearance (colours, hover glow, comment marker size).

`[thumbnail_panel]`
Thumbnail strip width, page numbers, and scrollbar visibility.

`[keybindings]`
Override any keybinding by specifying `action = "Key"` or `action = [ "Key1", "Key2" ]`.

`[mousebindings]`
Mouse button bindings, similar to keybindings.

2.2 Example snippet

```
[window]
menubar = false
accent = "#3daee9FF"

[statusbar]
visible = true
show_page_number = true
show_progress = true

[page]
bg = "#1e1e2e"
fg = "#cdd6f4"
```

For the full list of options see the configuration reference on the homepage.

3 Commands

Press `:` to open the command palette and run any command by name. Commands cover opening files, navigation, zoom, annotations, sessions, and more.

The full command reference is on the commands page of the homepage.

4 Lua API

LEKTRA embeds a Lua 5.x interpreter (build with `-DWITH_LUA=ON`). All APIs live under the global `lektra` table.

4.1 Configuration file

Place your Lua init file at `$HOME/.config/lektra/init.lua`. It is sourced at startup after the TOML config is applied.

4.2 Namespaces

`lektra.view` Access the current view, get one by id, or list all views; the view object exposes navigation, zoom, rotation, layout, search, text extraction, outline, and per-view event listeners (`register`, `register_once`, `unregister`, `clear_listeners`, `register_context_menu`).

`lektra.opt` Read/write runtime options (fit mode, layout mode, DPR, statusbar fields, ...).

lektra.cmd `register`, `unregister`, `execute`, `list`, `alias` — expose or invoke lektra commands from Lua.

lektra.keymap `set`, `get` — bind keys to Lua functions at runtime.

lektra.mousemap `set` — bind mouse buttons to Lua functions.

lektra.event `register`, `unregister`, `once`, `count`, and the `EventType` enum — global event listeners (file open/close, page change, zoom change, tab added/removed, …).

lektra.ui `message`, `messagebox`, `input`, `menu`, `picker`, `file_dialog`, `color_dialog`.

lektra.bookmarks `list` and per-bookmark helpers — programmatic bookmark management.

lektra.tabs `list`, `count`, `current`, `close`, `goto`, `first`, `last`, `next`, `prev`, `move_left`, `move_right`, `get_id`.

lektra.timer `new` — create one-shot / repeating timers driven by the Qt event loop.

lektra.utils `print`, `open_url`, `platform`.

lektra.version `major`, `minor`, `patch`, `full` — LEKTRA's version at runtime.

4.3 Quick examples

Bind a key to a Lua function. First register a lektra command whose implementation is your Lua callback, then bind keys to that command:

```lua
lektra.cmd.register("say_page", function()
    local v = lektra.view.current()
    lektra.ui.message("Page: " .. v:pageno())
end, "Print current page in the message bar")

lektra.keymap.set("say_page", { "<C-e>" })
```

React to a page change event:

```lua
lektra.event.register(lektra.event.EventType.OnPageChanged, function()
    local v = lektra.view.current()
    print("Now on page", v:pageno())
end)
```

Open a file in the current view:

```lua
local v = lektra.view.current()
v:open("/path/to/document.pdf")
```

4.4 Type stubs

LSP-friendly type stubs for the Lua API are installed at `$prefix/share/lektra/lua/` (Linux/macOS). Point your Lua language server at this directory for autocompletion.

```
// .luarc.json example
{
    "workspace.library": ["/usr/share/lektra/lua"]
}
```

5 LaTeX & SyncTeX

LEKTRA builds with SyncTeX support enabled by default. If your PDF was compiled with SyncTeX information (`-synctex=1` with `pdflatex`/`xelatex`/`lualatex`, or the equivalent flag in `latexmk`), you can:

- **Forward-search** from your editor into LEKTRA — jump from a source line to the corresponding location in the PDF. Editor integration snippets for Neovim, VS Code, Emacs, and TeXstudio are on the homepage.
- **Reverse-search** from LEKTRA back to your editor — click while holding the SyncTeX modifier (Ctrl by default) and LEKTRA runs your configured editor command with the source file and line.

Enable `behavior.auto_reload` in `config.toml` and re-running `latexmk` on a source change reloads the PDF automatically without losing your page position or zoom. Otherwise use `:file_reload` to reload manually.

6 Troubleshooting

6.1 DjVu files don't open

LEKTRA loads `libdjvulibre.so.21` at runtime. On Debian/Ubuntu install `libdjvulibre21`; on Arch install `djvulibre`; on Fedora `djvulibre`. No rebuild is needed — restart LEKTRA and DjVu will work.

6.2 SVGs look blurry or don't render vector graphics correctly

LEKTRA prefers `librsvg` + `libcairo` for SVG rendering; if either is missing it falls back to Qt's own SVG renderer, which handles a smaller subset of the spec. Install `librsvg2` (SONAME `.so.2`) for full quality.

6.3 No EXIF metadata in File Properties for images

Install `libexif` (SONAME `.so.12`). LEKTRA discovers it at runtime — no rebuild required.

6.4 Where LEKTRA keeps its files

`$HOME/.config/lektra/` `config.toml`, `init.lua`

`$HOME/.local/share/lektra/` recent files DB, sessions, bookmarks

`$HOME/.local/share/lektra/crashes/` crash logs (`crash_latest.log`)

`$HOME/.local/share/lektra/sessions/` JSON session files

6.5 LEKTRA crashed

A crash dialog appears with the log and a one-click "Report on GitHub" button that pre-fills an issue. You can also copy the log to the clipboard from that dialog, or find it later at `crash_latest.log` in the crashes directory. Debug and `RelWithDebInfo` builds include function names and file/line numbers in the trace; release builds show addresses only — running `addr2line -e /usr/bin/lektra` on those addresses resolves them if the binary keeps symbols.

7 Support

LEKTRA is developed by Dheeraj Vittal Shenoy and contributors, and is free and open-source software that will stay free forever. If it has been useful to you, consider supporting continued development through one of:

- Ko-fi — https://ko-fi.com/dheerajshenoy
- Liberapay — https://liberapay.com/dheerajshenoy
- GitHub Sponsors — https://github.com/sponsors/dheerajshenoy

Bug reports and feature ideas are welcome at the issue tracker on Codeberg (https://codeberg.org/lektra/lektra) or on GitHub (https://github.com/dheerajshenoy/lektra). `:donate` inside LEKTRA reopens the support dialog at any time.